

Rancangan Pengembangan Scrapebot Versi 2

Cakupan toko non-Shopify: ringkasan, alur kerja, perangkat, rencana kerja, dan keputusan teknis

Tanggal	5 Oktober 2026
Status	Rancangan, siap ditinjau
Dasar	Versi 1 (12 Agustus 2026), sudah berjalan
Dokumen sumber	docs/architecture.md, docs/roadmap.md, docs/decisions/0001-0003

Inti dokumen ini. Versi 1 sudah membaca toko Shopify secara tepat dan gratis. Toko lainnya bukan sulit *dijangkau*, melainkan sulit *dibaca*, karena tidak menyediakan data produk yang dapat dibaca mesin. Versi 2 menutup celah itu secara bertahap: cara termurah dan paling akurat dicoba lebih dahulu, peramban hanya dipakai bila perlu, dan model bahasa besar (LLM) menjadi upaya terakhir. Seluruhnya dibangun sendiri tanpa berlangganan layanan ScrapeGraph.

Daftar isi

1. Ringkasan
2. Latar belakang
3. Tujuan dan batasan
4. Alur kerja
5. Perangkat dan pustaka
6. Ekstraksi LLM tanpa ScrapeGraph
7. Keluaran
8. Rencana kerja
9. Etika dan penanganan data
10. Ringkasan keputusan teknis
11. Kesimpulan

1. Ringkasan

scrapebot mengunjungi situs web calon mitra ritel dan mengumpulkan bukti untuk keputusan penjualan grosir (*wholesale*): **apakah toko tersebut menjual produk rajutan, pada kisaran harga berapa, dan bagaimana cara menghubunginya?** Bot hanya mengumpulkan bukti. Keputusan tentang layak atau tidaknya sebuah toko tetap diambil oleh manusia.

Versi 1 bekerja dengan baik untuk toko Shopify karena Shopify menyediakan umpan produk (*product feed*) secara gratis. Untuk platform lain, versi 1 hanya membaca satu jenis data tertanam, yaitu JSON-LD. Toko yang tidak menyediakan JSON-LD berakhir dengan **no produk**, dan keluarannya tidak membedakan antara “tidak menjual rajutan” dan “katalognya tidak dapat dibaca”.

Versi 2 menambahkan beberapa cara untuk membaca situs non-Shopify. Cara-cara tersebut dicoba secara berurutan, mulai dari yang paling murah dan paling akurat hingga yang paling mahal. Peramban tanpa antarmuka (*headless browser*) dan LLM hanya digunakan untuk situs yang tidak dapat dibaca dengan cara yang lebih murah.

2. Latar Belakang

2.1 Temuan survei awal

Survei awal pada 12 Agustus 2026 mencakup 67 domain toko di Naples, Florida.

Temuan	Jumlah
Toko Shopify dengan umpan <code>/products.json</code> terbuka	26
Produk yang diperoleh dari umpan tersebut, lengkap dengan harga	6.167
Situs non-Shopify yang perlu dibaca dari HTML (situs khusus, Wix, WooCommerce, BigCommerce, Squarespace)	41
Beranda gagal diambil: HTTP 403 (H&M, Macy's, Four Seasons; ketiganya jaringan ritel nasional, bukan calon mitra)	3
Beranda gagal diambil: sertifikat TLS rusak	2

Tidak ada calon mitra yang hilang karena deteksi bot. Masalahnya adalah cakupan, bukan pemblokiran. Butik jarang memasang perlindungan antibiot; yang sering tidak mereka miliki adalah data produk yang dapat dibaca mesin.

2.2 Masalah versi 1 yang harus diperbaiki

No.	Masalah	Lokasi	Dampak
1	Pengambilan yang gagal (batas waktu, 429, 5xx) ikut disimpan di cache	<code>fetch.py</code>	Situs yang gagal sementara tidak pernah dicoba ulang kecuali cache dihapus manual
2	Situs tanpa JSON-LD berstatus ok dengan 0 produk	<code>sources.py</code>	Toko yang katalognya tak terbaca tampak seperti toko yang tidak menjual rajutan
3	Harga tidak disertai mata uang	<code>models.py</code>	Harga yang dilokalkan ke mata uang lain masuk ke kolom harga tanpa terdeteksi
4	Sebagian rancangan v1 belum dibangun: harga OpenGraph, penelusuran kedalaman 2, pemeringkatan teks tautan, domain paralel	<code>sources.py</code>	Cakupan lebih rendah daripada rancangan
5	Pola nomor telepon hanya mengenali format Amerika Serikat	<code>extract.py</code>	Kolom telepon kosong untuk toko di luar AS

3. Tujuan dan Batasan

3.1 Tujuan

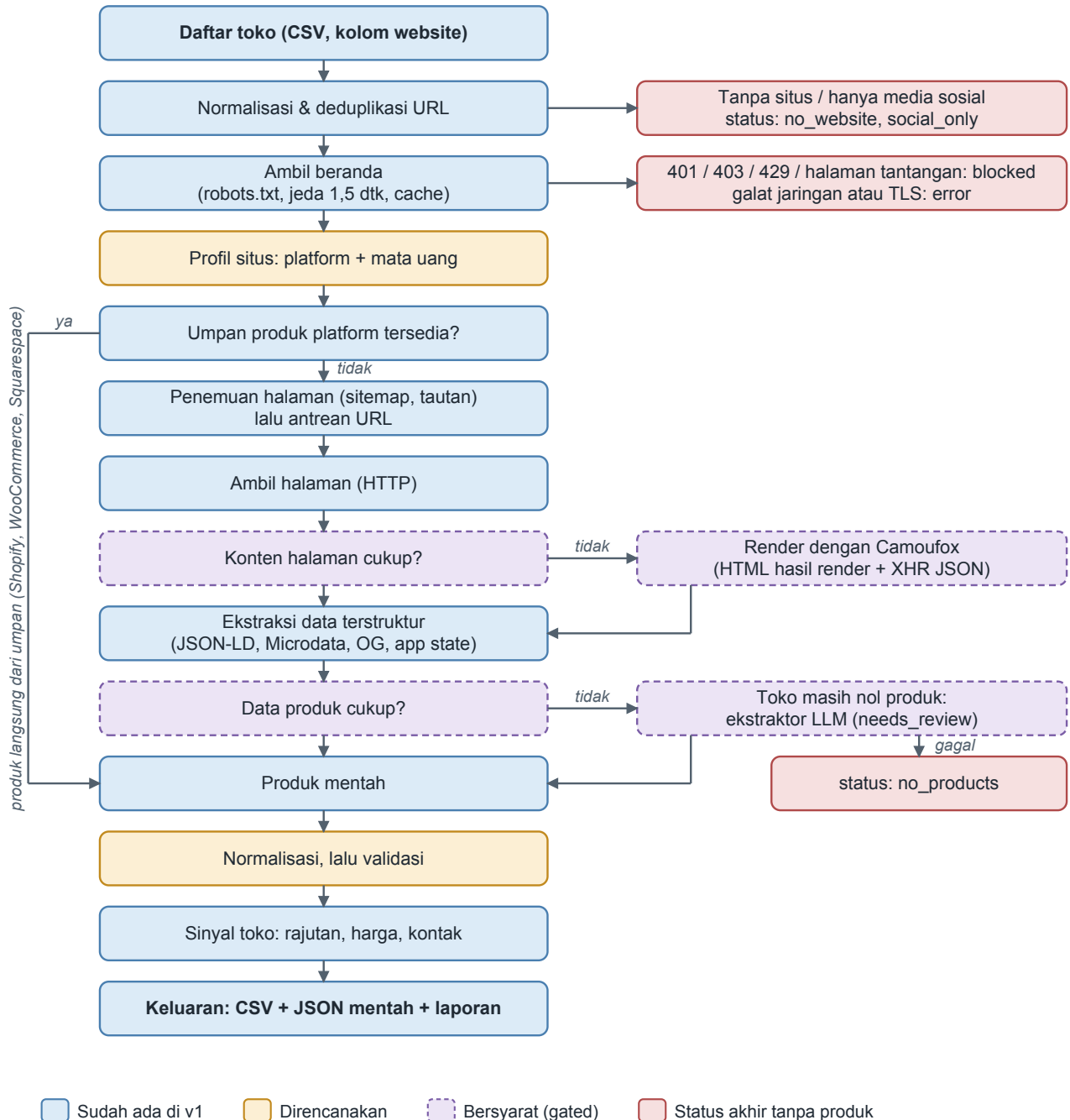
1. Membaca judul produk, harga, **dan mata uang** dari toko non-Shopify tanpa menulis selektor khusus untuk setiap situs.
2. Membuat setiap baris keluaran jujur tentang asal datanya (`source_used`) dan tingkat kepercayaannya (`needs_review`).
3. Mengeluarkan biaya (waktu peramban, token LLM) hanya untuk situs yang memerlukannya.
4. Memperbaiki masalah 1–3 versi 1 sebelum menambahkan lapisan baru.
5. Mempertahankan jaminan versi 1: setiap baris masukan menghasilkan tepat satu baris keluaran, seluruh pengujian berjalan tanpa jaringan, dan penilaian ulang tidak memerlukan pengambilan data ulang.

3.2 Di luar cakupan

- **Menilai atau memberi skor calon mitra.** Bot mengumpulkan bukti; manusia memutuskan.
- **Menembus perlindungan antibot.** Tidak ada penyelesaian tantangan Cloudflare atau CAPTCHA, tidak ada rotasi proksi untuk menghindari blokir, dan `robots.txt` selalu dipatuhi. Situs yang menantang atau memblokir dicatat sebagai `blocked`.
- **Masuk (login) ke situs mana pun** atau membaca konten yang tidak publik.
- **Mencari toko baru.** Daftar masukan tetap berasal dari Google Places.
- **Penjangkauan, pengiriman surel, atau integrasi CRM.**
- **Ekspor katalog lengkap.**
- **Basis data, API, atau dasbor,** selama bot masih dijalankan sekali per daftar toko.

4. Alur Kerja

Bentuk alur ini mengikuti alur kerja ScrapeGraph, tetapi dibangun dengan Python biasa dan pustaka sumber terbuka. Setiap toko berhenti pada tahap pertama yang menghasilkan produk. Halaman kontak dan halaman “tentang kami” selalu dikumpulkan, dari tahap mana pun produknya diperoleh.



Gambar 1. Alur kerja versi 2. Kotak bergaris putus-putus hanya dibangun bila hasil survei Tahap 1 membenarkannya.

4.1 Resolusi masukan

Bot membaca CSV masukan; hanya kolom `website` yang wajib. Kolom lain diteruskan ke keluaran tanpa perubahan. URL dinormalisasi dan duplikat `www.` digabung menjadi satu toko, tetapi setiap baris asli tetap mendapat baris keluarannya sendiri. Baris tanpa situs diberi status `no_website`, dan baris yang hanya berisi tautan media sosial diberi status `social_only`.

4.2 Pengambilan halaman

Semua permintaan HTTP melewati `Fetcher`, satu-satunya modul yang mengakses jaringan:

- `robots.txt` diperiksa untuk setiap URL, dan jeda 1,5 detik dijaga antara permintaan ke domain yang sama.
- Respons disimpan di cache, dan permintaan yang gagal dicoba ulang dengan jeda bertahap.
- Sertifikat TLS yang rusak dicoba sekali tanpa verifikasi, lalu baris tersebut ditandai `ssl_bypassed` agar kelemahan itu terlihat.
- Header `Accept-Language` sengaja tidak dikirim. Jika dikirim, Shopify Markets melokalkan harga ke negara peminta (harga dalam rupiah pernah teramati).
- *Direncanakan*: pengambilan yang gagal tidak lagi disimpan di cache.

4.3 Beranda dan profil situs

Respons 401, 403, atau 429 menghasilkan status `blocked`, sedangkan kegagalan jaringan atau TLS menghasilkan status `error`; keduanya menghentikan proses untuk toko tersebut. Halaman tantangan Cloudflare juga akan dikenali sebagai `blocked`. Platform dideteksi dari penanda di beranda.

Direncanakan: mata uang dideteksi secara berurutan dari `Shopify.currency.active`, `priceCurrency` pada JSON-LD, lalu kolom mata uang pada umpan platform.

4.4 Umpan produk platform

Umpan langsung mengembalikan judul dan harga, sehingga produknya langsung masuk ke daftar produk mentah. **Halaman produk tidak diambil satu per satu.** Mengambil 6.167 halaman produk dengan jeda 1,5 detik per domain akan memakan waktu berjam-jam dan membebani situs calon mitra tanpa manfaat tambahan.

Umpan	Alamat	Status
Shopify	<code>/products.json</code> , 250 produk per halaman, maksimal 8 halaman	Sudah ada
WooCommerce Store API	<code>/wp-json/wc/store/v1/products</code> ; harga berupa teks dalam satuan terkecil (sen)	Direncanakan
Squarespace	<code>?format=json</code> ; bentuk kolom harga berbeda antarversi templat	Direncanakan
Lightspeed eCom	<code>?format=json</code> (diverifikasi saat survei)	Direncanakan

4.5 Penemuan halaman dan antrean URL

Tahap ini hanya berjalan bila tidak ada umpan yang menghasilkan produk. Bot membaca `sitemap.xml` (termasuk indeks sitemap satu tingkat) atau, bila tidak ada, menelusuri tautan internal beranda. URL disaring, dideduplikasi, dan diurutkan: halaman kontak, “tentang kami”, dan grosir didahulukan, sehingga toko dengan ribuan URL produk tidak menggeser halaman kontak keluar dari anggaran 25 halaman.

Direncanakan: sitemap khusus produk (Wix, Yoast, WordPress), kanonisasi URL, dan penelusuran kedalaman 2.

4.6 Pemeriksaan konten dan render peramban (bersyarat)

Setiap halaman diperiksa: **apakah ada konten yang dapat dibaca?** Halaman yang kosong atau hanya berupa kerangka JavaScript dirender dengan **Camoufox**. HTML hasil render diteruskan ke ekstraksi data terstruktur, dan respons JSON yang dimuat halaman serta tampak seperti data produk ikut ditangkap beserta URL-nya sebagai bukti. Keputusan diambil **per halaman**, maksimal 10 halaman per toko. Lokal peramban dikunci ke `en-US`, dan peramban tunduk pada pemeriksaan `robots.txt` serta jeda yang sama dengan HTTP.

4.7 Ekstraksi data terstruktur

Sumber	Status
JSON-LD Product	Sudah ada
Microdata, OpenGraph product:price:amount, RDFa (melalui <code>extract</code>)	Direncanakan
State aplikasi di <code><script>: __NEXT_DATA__, __NUXT__, window.__INITIAL_STATE__</code>	Direncanakan
JSON XHR yang ditangkap peramban	Bersyarat

4.8 Ekstraktor LLM (bersyarat)

Tahap ini hanya berjalan untuk toko yang **masih nol produk** setelah semua tahap sebelumnya. Rinciannya dijelaskan pada bagian 6.

4.9 Normalisasi dan validasi (direncanakan)

Normalisasi menyeragamkan produk dari semua sumber: harga menjadi bilangan desimal (satuan sen WooCommerce dikonversi), mata uang menjadi kode ISO 4217, judul dibersihkan, dan produk ganda digabung. **Validasi** mensyaratkan judul tidak kosong, harga positif atau tidak ada sama sekali, mata uang diketahui (atau baris ditandai), dan untuk produk dari LLM, `source_url` harus termasuk halaman yang dikunjungi. Produk yang gagal validasi dibuang dan dihitung dalam laporan, karena data salah yang tersembunyi lebih buruk daripada celah yang terlihat.

4.10 Sinyal toko

Dari produk yang lolos validasi dan halaman yang terkumpul, bot menghitung jumlah dan persentase produk rajutan beserta hingga lima contoh judul, rentang harga rajutan dan rentang harga seluruh toko, kontak (surel, telepon, Instagram, Facebook), halaman grosir, cuplikan “tentang kami”, dan penanda jaringan ritel nasional. *Direncanakan*: rentang harga dihitung dalam satu mata uang saja.

5. Perangkat dan Pustaka

5.1 Yang dipakai

Komponen	Pilihan	Keterangan
Orkestrasi	Perulangan Python biasa	Rute ditentukan oleh aturan yang pasti, bukan oleh model
HTTP	<code>requests</code> melalui <code>Fetcher</code>	Sudah berjalan; cache, jeda, dan pemeriksaan robots sudah ada
HTML	BeautifulSoup + <code>lxml</code>	Sudah dipakai di v1
Data terstruktur	<code>extract</code>	Baru; membaca JSON-LD, Microdata, OpenGraph, dan RDFa sekaligus
Peramban	Camoufox	Opsional dan bersyarat; Firefox dengan sidik jari realistis dan API yang kompatibel dengan Playwright
Model data	Pydantic	Baru; skema keluaran LLM dan tahap validasi
LLM	Anthropic SDK (Claude Haiku 4.5) atau model lokal melalui Ollama	Opsional dan bersyarat; dipanggil langsung tanpa kerangka kerja
Pengujian	<code>pytest</code>	Seluruh pengujian tanpa jaringan, memakai fixture dari situs nyata

5.2 Yang ditunda atau tidak dipakai

Pilihan	Keputusan	Alasan
LangGraph	Ditunda	Rute alur ditentukan oleh percabangan yang pasti, dan kebutuhan melanjutkan proses sudah terpenuhi oleh cache berkas. Ditinjau ulang bila ada tahap yang memerlukan LLM untuk memilih langkah berikutnya. Bila setiap tahap ditulis sebagai fungsi murni, pemindahan ke LangGraph kelak cukup murah
httpx	Ditunda	Bermanfaat untuk async dan HTTP/2 bila domain diambil secara paralel. <code>Fetcher</code> sudah memiliki transport yang dapat diganti
PostgreSQL	Ditunda	Belum ada kebutuhan riwayat antarjalan. Bila bot dijalankan berkala, mulai dengan SQLite sebagai penulis keluaran kedua
Redis	Tidak diperlukan	Bot berjalan sebagai satu proses dengan cache berkas
API ScrapeGraph	Ditolak	Berbayar per permintaan; tekniknya dapat dibangun sendiri (bagian 6)
Pustaka ScrapeGraphAI	Opsional sebagai pembanding	Sumber terbuka dan gratis, tetapi menarik LangChain dan Playwright-Chromium serta mengirim setiap potongan halaman ke LLM
Botasaurus	Ditolak	Menduplikasi <code>Fetcher</code> (cache, coba ulang, jeda) dan tumpang tindih dengan Camoufox
Playwright-Chromium	Ditolak	Chromium tanpa antarmuka mudah dikenali; Camoufox memakai API yang sama
Crawl4AI, Firecrawl	Tidak dipakai	Membawa penelusur dan peramban sendiri; Firecrawl juga memerlukan server sendiri
CrewAI, Agno	Tidak diperlukan	Tidak ada kebutuhan sistem multiagen

5.3 Berkas dependensi

Dependensi dipisah agar perjalanan dasar tetap ringan. Mesin tanpa peramban atau kunci API tetap dapat menjalankan seluruh tahap kecuali render peramban dan LLM, dan tahap yang dilewati dicatat dalam laporan.

Berkas	Isi	Diperlukan untuk
<code>requirements.txt</code>	Dependensi yang ada + <code>extract</code> , <code>pydantic</code>	Setiap perjalanan
<code>requirements-browser.txt</code>	<code>camoufox[geoip]</code> , lalu <code>camoufox</code> <code>fetch</code> sekali	Render peramban
<code>requirements-llm.txt</code>	<code>anthropic</code> (atau Ollama terpasang lokal)	Ekstraktor LLM
<code>.env.example</code>	<code>ANTHROPIC_API_KEY=</code>	Ekstraktor LLM dengan Claude

6. Ekstraksi LLM tanpa ScrapeGraph

ScrapeGraphAI tersedia dalam dua bentuk: **API terkelola** yang membayar per permintaan, dan **pustaka sumber terbuka** berlisensi MIT yang gratis. Proyek ini tidak memakai keduanya. Yang diambil hanyalah tekniknya, yaitu menjelaskan *apa* yang ingin diambil melalui skema, bukan *di mana* letaknya dalam HTML.

6.1 Cara kerja

Inti `SmartScraperGraph` pada dasarnya terdiri atas lima langkah. Dua langkah pertama sudah ada di bot ini, sehingga yang perlu dibangun hanya langkah 3–5, diperkirakan sekitar 100–150 baris kode.

Langkah	Isi	Status
1	Ambil HTML halaman	Sudah ada (<code>Fetcher</code>)
2	Ubah HTML menjadi teks	Sudah ada (<code>html_to_text</code>)
3	Potong teks menjadi beberapa bagian bila terlalu panjang	Dibangun
4	Kirim ke LLM bersama skema Pydantic	Dibangun
5	Gabungkan hasil, lalu terapkan aturan bukti	Dibangun

6.2 Skema dan aturan bukti

- Skema keluaran: `store_type, products[{{title, price, currency, source_url}}, brands_carried, has_wholesale_page`.
- Produk yang `source_url`-nya bukan salah satu halaman yang dikirim akan dibuang.
- Setiap baris dari tahap ini diberi `needs_review = True`.
- Versi prompt, nama model, jumlah token, dan respons mentah disimpan di JSON mentah.

6.3 Pilihan model

	Claude Haiku 4.5 (API)	Model lokal melalui Ollama
Biaya	Per token; volumenya kecil karena hanya untuk toko yang masih nol produk	Gratis
Akurasi	Lebih tinggi	Lebih rendah; tetap layak karena semua hasil ditinjau manusia
Kebutuhan	Kunci API di <code>.env</code>	Mesin dengan memori yang cukup untuk model 7–8B (mis. Qwen atau Llama)

6.4 Keunggulan dibanding memakai pustaka

- Tidak menarik LangChain dan Playwright-Chromium.
- Dapat diuji tanpa jaringan dengan fixture respons LLM yang direkam.
- Hanya halaman pilihan bot yang dikirim, sehingga token lebih hemat.
- Penyedia model dapat diganti tanpa mengubah alur.

7. Keluaran

Setiap penjalanan menghasilkan tiga berkas, sama seperti versi 1, dengan beberapa tambahan.

- `data/out/<input>-enriched.csv`: satu baris untuk setiap baris masukan.
- `data/raw/<domain>.json`: semua yang terkumpul, ditambah jejak tahap, sumber mata uang, URL XHR, dan catatan pemanggilan LLM.
- `data/out/run-report.md`: jumlah per status, ditambah tabel cakupan per sumber, daftar `no_products` dan `needs_review`, produk yang gugur validasi, tahap yang dilewati, dan total token LLM.

7.1 Kolom baru

Kolom	Arti
<code>currency</code>	Kode ISO mata uang harga pada baris ini; kosong bila tidak diketahui
<code>currency_mixed</code>	<code>True</code> bila produk toko muncul dalam lebih dari satu mata uang; kolom harga memakai mata uang mayoritas
<code>source_used</code>	Diperluas menjadi <code>shopify_feed</code> , <code>woocommerce_api</code> , <code>squarespace_json</code> , <code>structured_data</code> , <code>browser_render</code> , <code>browser_xhr</code> , <code>llm</code> , <code>none</code>
<code>layers_tried</code>	Seberapa jauh toko menuruni alur, mis. <code>feed</code> , <code>structured</code> , <code>browser</code>
<code>needs_review</code>	<code>True</code> bila produk berasal dari LLM; baris ini diperiksa manusia lebih dahulu

7.2 Status

Status	Arti
<code>ok</code>	Produk ditemukan
<code>no_products</code>	<i>Baru.</i> Situs terbaca, tetapi tidak ada tahap yang menemukan katalog. Pada v1, toko seperti ini keliru berstatus <code>ok</code>
<code>js_required</code>	Memerlukan peramban yang belum terpasang atau belum dibangun
<code>blocked</code>	401, 403, 429, atau halaman tantangan; tidak ditembus
<code>ssl_bypassed</code>	Dibaca tanpa verifikasi TLS karena sertifikatnya rusak
<code>error</code>	Kegagalan jaringan, galat HTTP lain, atau dilarang oleh <code>robots.txt</code>
<code>no_website</code> , <code>social_only</code>	Dilewati pada tahap resolusi

8. Rencana Kerja

Setiap tahap diakhiri dengan gerbang keputusan. Tahap berikutnya hanya dimulai bila angka dari tahap sebelumnya membenarkannya.

Tahap	Pekerjaan	Gerbang
0. Perbaikan v1	Pengambilan gagal tidak disimpan di cache; status <code>no_products</code> ; mata uang pada produk dan deteksinya; rentang harga satu mata uang; deteksi halaman tantangan; <code>Fetcher.allowed()</code> untuk peramban	Semua pengujian lama lulus; setiap perbaikan punya pengujian baru
1. Survei	Penjalaran baca-saja atas ±41 situs non-Shopify. Untuk setiap situs, catat tahap mana yang akan berhasil dan ukur sinyal “konten cukup?”	Matriks cakupan ditinjau; menentukan apakah Tahap 3 dan 4 dikerjakan
2. Umpan, ekstraksi, normalisasi, validasi	Umpan WooCommerce dan Squarespace; <code>extract</code> dan state aplikasi; sitemap produk; kanonisasi URL; tahap normalisasi dan validasi; kekurangan v1 nomor 4	Fixture dari situs nyata untuk setiap pengurai baru; cakupan diukur ulang
3. Render peramban (bersyarat)	Pemeriksaan konten per halaman; Camoufox di balik antarmuka yang dapat disuntikkan; penangkapan JSON XHR	Hanya bila survei menunjukkan minimal 5 situs calon mitra yang hanya dapat dibaca dengan peramban
4. Ekstraktor LLM (bersyarat)	Panggilan langsung dengan skema Pydantic; aturan bukti; <code>needs_review</code> ; pencatatan token; respons rekaman sebagai fixture	Hanya bila minimal 5 situs calon mitra masih <code>no_products</code> setelah Tahap 3

Tahap	Pekerjaan	Gerbang
5. Rilis	Penjalanan penuh atas daftar calon mitra; pembaruan README; VERSION 2.0.0 dan CHANGELOG.md	Laporan menampilkan cakupan per sumber dan jumlah <code>needs_review</code>

Ambang 5 situs setara dengan sekitar 10% daftar non-Shopify. Di bawah angka itu, memeriksa sisa situs secara manual lebih murah daripada membangun dan merawat tahap baru. Seluruh pengujian tetap berjalan tanpa jaringan: peramban dan LLM berada di balik antarmuka yang dapat disuntikkan, dan pengujian memakai tiruan yang mengembalikan HTML hasil render dan respons LLM yang telah direkam.

9. Etika dan Penanganan Data

- `robots.txt` dipatuhi, jeda 1,5 detik per domain, hanya halaman publik, dan tanpa login. Aturan yang sama berlaku untuk peramban.
- Camoufox dipakai untuk merender JavaScript, bukan untuk menembus tantangan. Halaman tantangan dikenali dan dicatat sebagai `blocked`.
- Lokal peramban dikunci ke `en-US` dan mata uang selalu dicatat, sehingga harga yang dilokalkan terlihat, bukan diam-diam salah.
- Hanya teks halaman publik yang dikirim ke LLM. Kunci API disimpan di `.env` dan tidak pernah dimasukkan ke repositori.
- Menghubungi toko berada di luar alat ini. Apakah dan bagaimana toko dihubungi adalah keputusan komersial dan kepatuhan (CAN-SPAM) bagi operator.

Mengapa perlindungan antibot tidak ditembus. Tantangan, respons 403, atau aturan `robots.txt` adalah penolakan eksplisit pemilik situs terhadap akses otomatis. Toko-toko ini adalah calon mitra grosir, sehingga risiko reputasi dan hukumnya tidak sepadan. Penembusan juga memerlukan perawatan terus-menerus serta biaya pemecah CAPTCHA atau proksi, padahal data survei menunjukkan kebutuhannya hampir nol. Keputusan ini ditinjau ulang bila daftar kota berikutnya menunjukkan banyak butik (bukan jaringan ritel) yang terblokir; yang diperiksa lebih dahulu adalah laju permintaan dan jenis alamat IP.

10. Ringkasan Keputusan Teknis

No.	Keputusan	Inti	Status
0001	Akuisisi berlapis, LLM terakhir	Meniru bentuk alur ScrapeGraph tanpa pustaka maupun API-nya. Umpan langsung menjadi produk; LLM hanya untuk toko yang masih nol produk	Diterima
0002	Camoufox hanya untuk render	Peramban hanya untuk halaman yang gagal pemeriksaan konten; tantangan dan 401/403/429 dicatat sebagai <code>blocked</code> ; Botasaurus ditolak	Diterima
0003	Orkestrasi Python biasa; keluaran berkas	Tanpa LangGraph, PostgreSQL, atau Redis untuk saat ini. Ditinjau ulang bila bot dijalankan berkala dan memerlukan riwayat	Diterima

11. Kesimpulan

Versi 1 sudah menjawab pertanyaan utama untuk toko Shopify, sekitar 40% dari daftar, secara tepat dan gratis. Sisa daftar bukan sulit dijangkau, karena tidak ada calon mitra yang terblokir, melainkan sulit dibaca, karena situs-situs itu tidak menyediakan umpan produk.

Versi 2 menutup celah tersebut secara berlapis, dimulai dari yang termurah:

- **Umpan platform dan data terstruktur** diperkirakan memulihkan sebagian besar toko non-Shopify tanpa biaya tambahan.
- **Peramban** menangani situs yang hanya dapat dibaca dengan JavaScript, dan hanya untuk halaman yang memerlukannya.
- **LLM** menangani sisanya dengan teknik ScrapeGraph yang dibangun sendiri, memakai Claude atau model lokal, dan setiap barisnya ditandai untuk ditinjau.

Survei pada Tahap 1 akan mengukur berapa banyak situs yang dipulihkan oleh setiap tahap. Render peramban dan LLM hanya dibangun bila angka tersebut membenarkannya.

Tiga prinsip yang tidak berubah: bot mengumpulkan bukti dan manusia yang menilai; setiap baris menyebutkan asal datanya; dan tidak ada kontrol akses yang ditembus.